Day 30

Promise Chaining .then() calls:

What is Promise Chaining?

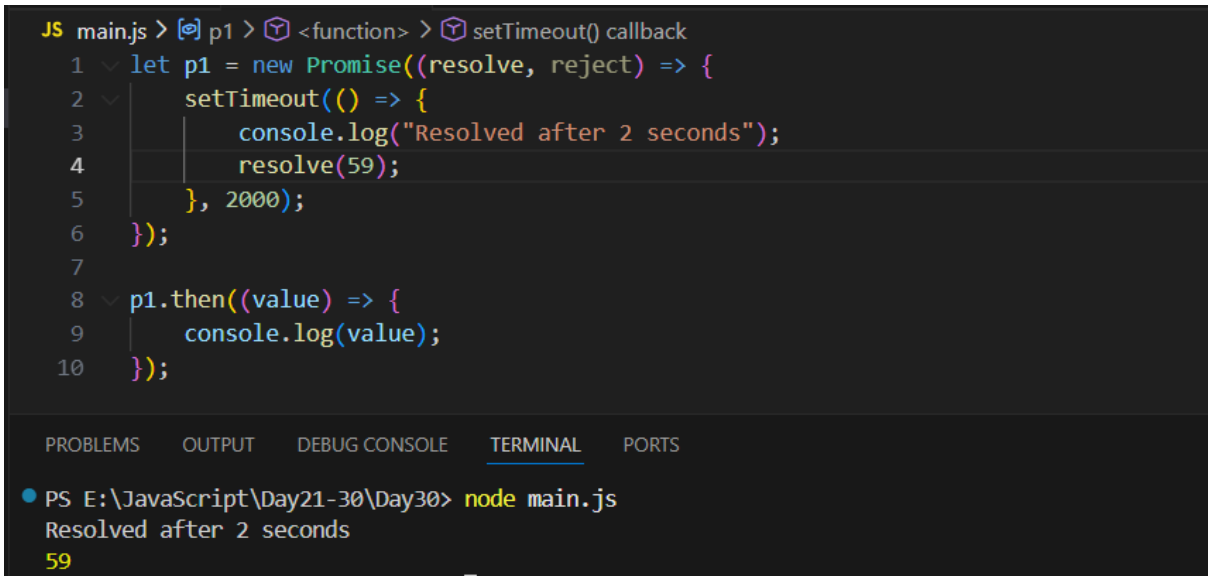
Promise chaining means using multiple .then() one after another, where the output of one .then() becomes the input of the next.

Each .then() returns a new Promise, allowing you to run steps in order.

Why use Promise chaining?

- To perform tasks step-by-step
- To avoid callback hell
- To make async code readable and organised

Example: a very basic example of promise with ".then()"



Above code creates a Promise (p1) that waits 2 seconds and then prints "Resolved after 2 seconds" before successfully completing with the value 59. When the promise finishes, the .then() function runs and prints that resolved value (59). In simple terms: "Wait 2 seconds, show a message, return 59, then print 59."

Example: promise chaining basics

```
11
12  let p1 = new Promise((resolve, reject) => {
13      setTimeout(() => {
14          console.log("Resolved after 2 seconds");
15          resolve(59);
16      }, 2000);
17  });
18
19  p1.then((value) => {
20      console.log(value);
21      let p2 = new Promise((resolve, reject) => {
22          resolve("Promise p2 resolved");
23      });
24      return p2;
25  }).then((value) => {
26      console.log("We are done");
27      console.log(value);
28  });
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day21-30\Day30> node main.js
Resolved after 2 seconds
59
We are done
Promise p2 resolved
```

This code shows promise chaining. The first promise (p1) waits 2 seconds, prints "Resolved after 2 seconds", and resolves with 59. The first .then() receives that value, prints 59, and then creates and returns another promise (p2), which immediately resolves with "Promise p2 resolved". Because it returns a promise, the next .then() waits for p2 to finish, then prints "We are done" followed by the message from p2. In short: wait 2 seconds → print 59 → run another promise → print final messages.

Example: another example of promise chaining.

```
12  let p1 = new Promise((resolve, reject) => {
13      setTimeout(() => {
14          console.log("Resolved after 2 seconds");
15          resolve(59);
16      }, 2000);
17  });
18
19  p1.then((value) => {
20      console.log(value);
21      let p2 = new Promise((resolve, reject) => {
22          setTimeout(() => {
23              resolve("Promise 2 resolved");
24          }, 2000);
25      });
26      return p2;
27  }).then((value) => {
28      console.log("We are done");
29      console.log(value);
30  });
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day21-30\Day30> node main.js
Resolved after 2 seconds
59
We are done
Promise 2 resolved
```

This code shows two promises running one after another. First, p1 waits 2 seconds, prints "Resolved after 2 seconds", and resolves with 59. Inside the first .then(), that value (59) is printed, and a second promise p2 is created. p2 also waits 2 seconds before resolving with "Promise 2 resolved". Because

the .then()returnsp2, the next .then()` waits for it to finish, then prints "We are done" and the final message.

Attaching Multiple Handlers to a Promise:

You can attach more than one .then() or .catch() to the same Promise. Each handler will run independently when the Promise is resolved or rejected.

The key point: Every .then() attached to the same Promise runs separately. They do NOT depend on each other.

Example: a very basic promise created in JS

Can we write p1.then() and define a new work? Yes, we can do so, as shown below:

Each p.then() works independently.

--The End--